
Contents

Procedure in RISC-V	1
Invocazione	1
Side effect dell'utilizzo delle procedure	2
Sovrascrittura dei registri	2
Procedure annidate	2
Parametri numerosi	2
Record di attivazione di una procedura	2
I registri s1-s11	3
Convenzioni nel salvataggio di registri	3
Fasi di invocazione di una procedura	4

Procedure in RISC-V

Chiamante - Mettere i parametri in un luogo accessibile alla procedura - Trasferire il controllo alla procedura

Chiamato(procedura) - Acquisire le risorse necessarie per l'esecuzione della procedura - Eseguire il compito richiesto - Mettere il risultato in un luogo accessibile al programma chiamante - Restituire il controllo al punto di origine (la stessa procedura può essere chiamata in differenti punti di un programma)

Invocazione

jal x1, indirizzoProcedura

Salta ad indirizzoProcedura e memorizza il valore di PC+4 nel registro x1(return address). Cioè l'istruzione dopo **jal**

Come pseudoistruzione si può omettere x1.

jalr rd, offset(rs1) Jump and link register.

Salta ad un indirizzo calcolato come offset + rs1 e memorizza il valore di PC+4 nel registro x1(return address). Cioè l'istruzione dopo **jalr**

jalr viene rappresentata con il formato I

Per entrambe le istruzioni, il return address viene usato quando alla fine di una procedura vogliamo tornare all'esecuzione del codice dopo la chiamata alla procedura.

In questo modo, dopo un **jal** ad un'etichetta, possiamo tornare alla esecuzione del codice attraverso **jalr** che accetta come valore un registro.

jalr viene spesso usata per ritornare al chiamante di una funzione, usando come registro x0, in quanto non ci serve il valore di PC+4.

Esempio: **jalr x0, 0(x1)**

Viene codificata in formato I

Side effect dell'utilizzo delle procedure

Sovrascrittura dei registri

Quando non ci bastano i registri per salvare i nostri numeri e alcuni sarebbero sovrascritti nella procedura, salviamoli temporaneamente in memoria nella funzione chiamata, la procedura chiamata lo usa e prima di ritornare alla funzione chiamante, ricarichiamo il valore iniziale dalla memoria nel registro.

Procedure annidate

Se chiamiamo più procedure annidate tra loro (una procedura ne chiama un'altra), allora il valore di *r* a viene sovrascritto. L'unico modo per preservare il valore di *r* a è salvarlo in memoria e poi richiamarlo quando necessario.

Parametri numerosi

Può succedere che dobbiamo passare troppi parametri ad una procedura, allora al posto di usare i registri, possiamo caricarli in memoria e caricarli progressivamente quando ci servono.

Record di attivazione di una procedura

Ogni procedura nello stack ha il proprio **record di attivazione** che contiene i valori salvati nello stack dalla funzione. Il processo di trasferimento in memoria delle variabili viene chiamato **register spilling**.

Quando la funzione termina, il record di attivazione viene cancellato.

Anche il main ha il suo record di attivazione.

L'inizio del record di attivazione della procedura in esecuzione viene salvato nel frame pointer.

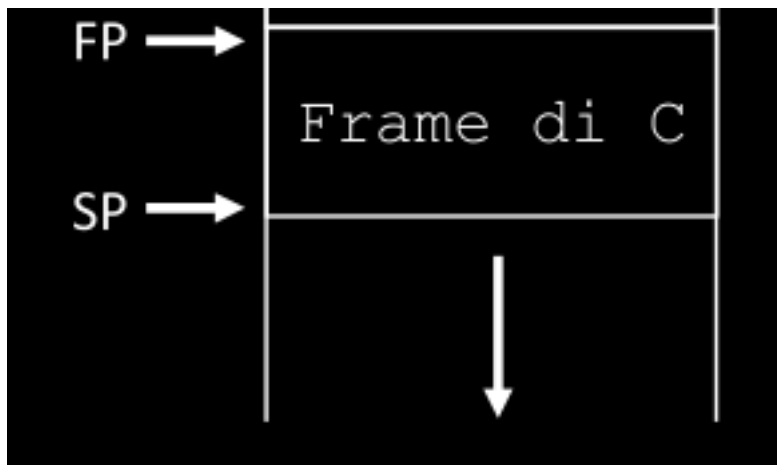


Figure 1: alt text

Il frame pointer, come lo stack pointer, va gestito nel codice(non è automatico). Tuttavia il frame pointer non cambia durante l'esecuzione della procedura. Possiamo quindi usarlo per selezionare blocchi di memoria.

Può essere omesso per complicare meno il codice.

I registri s1-s11

Vanno usati per il salvataggio di variabili che potrebbero essere sovrascritte

Convenzioni nel salvataggio di registri

Ci sono delle convenzioni per decidere chi salva i registri che andrebbero sovrascritti.

Questo problema nasce perchè la procedura chiamata sa i suoi registri importanti, la procedura chiamante sa quali registri userà.

Bisogna garantire il minimo salvataggio dei registri, cioè non entrambe le funzioni devono salvare gli stessi registri.

Inoltre, la funzione chiamante non sa che registri userà la funzione chiamata quindi potrebbe fare salvataggi inutili.

Allo stesso modo la funzione chiamata non sa se i registri che usa, vanno salvati oppure no.

Per risolvere questi dubbi vengono create delle **convenzioni**:

I registri:

-
- a0-a7 e t0-t6 possono essere modificati dal chiamato senza necessità di ripristino da parte sua. Il chiamante si dovrà occupare, se necessario, del salvataggio e del ripristino dei valori
 - ra, sp, gp, tp, fp e s1-s11 se modificato dal chiamato devono essere salvati e ripristinati prima del ritorno al chiamante. Il chiamante non è tenuto al salvataggio e al ripristino

Una procedura potrebbe dover prevedere entrambe le cose perchè può essere chiamante e chiamato.

Anche se sappiamo che alcuni registri non sono importanti dobbiamo comunque seguire le convenzioni

Usando questa convenzione, e usando i registri in maniera corretta(cioè usando il primo blocco di registri solo nelle procedure chiamate, il secondo blocco nel codice chiamante. Ovviamente se il numero di registri bastano, altrimenti saremo costretti ad operare nello stack), riusciamo a minimizzare le operazioni su memoria centrale.

Fasi di invocazione di una procedura

Per richiamare una procedura dobbiamo procedere scrivendo 7 fasi(anche i compilatori nelle chiamate a funzioni quando convertono fanno questi passi):

1. Pre-chiamata cioè salvare i registri necessari secondo le convenzioni(se necessario) e scrivere gli argomenti nei registri a0-a7 secondo l'ordine che la procedura impone(primo argomento in a0, secondo in a1, ...). Se i registri non bastano, useremo lo stack(anche qui seguiamo l'ordine della procedura)
2. Invocazione della procedura, eseguiamo il `jal`

I passi 1-2 vengono fatti dal chiamante

3. Prologo del chiamato, nella procedura, se necessario e secondo le convenzioni, salviamo i registri che andiamo ad usare. I registri vanno salvati in quest'ordine:
 - Registro ra, va salvato se la procedura non è foglia(cioè va salvato se la procedura è annidata), questo perchè il return address ci serve per tornare chiamante
 - Registro fp e sp, va salvato se usiamo lo stack all'interno della procedura(il record di attivazione). Dopo averlo salvato possiamo inizializzarne uno nuovo per la procedura(cioè settarlo `sp+offset`). Il registro sp va decrementato di un `offset+4` perchè dobbiamo saltare la word della precedente chiamata. Per salvare il record di attivazione della funzione. Questo offset è la dimensione del record di attivazione della funzione(nei nostri esempi è statica, ma potrebbe incrementare dinamicamente).

Per il resto seguiamo le convenzioni.

-
4. Corpo nella procedura, l'implementazione della procedura.
 5. Epilogo del chiamato, scriviamo i valori di ritorno in a0-a1, ripristiniamo i registri che avevamo salvato nel prologo. Per il registro sp, se abbiamo usato lo stack, basta incrementarlo dello stesso offset del prologo.
 6. Ritorno al chiamante, la chiamata `jalr x0, 0(x1)`

I passi 3-6 vengono fatti dal chiamato

7. Post-chiamata, usiamo il risultato della funzione a0-a1 e poi ripristiniamo i valori. Se facessimo il contrario, il risultato andrebbe perso.